

Computer Science Engineering BSc (2021) – Final Exam Answers

This document provides comprehensive answers to all 15 exam topics following the format from the GitHub repository example.

Question 1: Embedded Systems & Programming Concepts

Part A: Embedded Systems

Embedded Systems Definition

An embedded system is a specialized computer system that combines a processor, memory, and input/output peripheral devices with a dedicated function within a larger mechanical or electronic system. Unlike general-purpose computers, embedded systems are task-specific and designed to perform particular operations with high precision and efficiency.

Main Characteristics

Dedicated Functionality: Designed for specific applications (automotive control, medical devices, industrial automation) **Real-Time Operation:** Must respond within strict timing constraints **Resource Constraints:** Limited memory, processing power, and energy compared to general-purpose computers **Reliability and Stability:** Must operate reliably for extended periods, especially in safety-critical applications **Physical World Interaction:** Interface with environment through sensors and actuators

Control Units of Embedded Systems

Integration Levels:

- Small-Scale Integration (SSI): Tens of transistors
- Medium-Scale Integration (MSI): Hundreds of transistors
- Large-Scale Integration (LSI): Thousands of transistors
- Very Large-Scale Integration (VLSI): Hundreds of thousands of transistors
- Ultra-Large-Scale Integration (ULSI): Millions of transistors

System on Chip (SoC): Complete system with processors, memory, peripherals on single chip. Can be ASIC or FPGA-based.

Control Unit Types:

- Microcontrollers: Integrate processor, memory, I/O on single chip
- Microprocessors: Higher computational power, require external components
- Digital Signal Processors (DSPs): Specialized for signal processing

Interfaces and Peripherals:

- UART: Serial communication
- I2C: Short-distance inter-chip communication
- SPI: High-speed synchronous communication
- USB: Standardized connectivity
- GPIO: Flexible interfacing
- ADC/DAC: Signal conversion
- PWM: Motor control and analog signal generation

Programming Techniques:

- Control Loop Programming: Simple loops calling subroutines
- Interrupt-Driven: Main and secondary loops with interrupt handling
- RTOS: Task scheduling and resource management for complex systems
- Bare-Metal: Direct hardware programming without OS
- State Machine: Finite state machine implementation

Implementing User Interaction

Input Methods: Physical buttons, touchscreens, keypads, rotary encoders, voice/gesture recognition, remote controls

Output Methods: LEDs, LCD/OLED displays, seven-segment displays, audio feedback, haptic feedback

Communication: Bluetooth, Wi-Fi, Zigbee, web interfaces, command-line interfaces

Part B: Program Units

A program unit is a sequence of lines organized as statements, comments, and directives. Types include:

- Main Program: Entry point
- Modules: Self-contained code grouping
- Block Data Program Units: Data initialization (Fortran)
- External Function Subprograms: Return values
- External Subroutine Subprograms: Perform operations without direct return

Subprograms

A subprogram is an instruction sequence invoked from remote locations, returning control after completion. Called subroutines, procedures, functions (high-level) or methods, constructors (OOP).

Benefits: Code reusability, modularity, abstraction, independent testing

Parameter Evaluation

Maps formal (in specification) and actual (in calls) parameters.

Issues:

1. **Assignment:** Positional (by order) or named (explicit parameter names)
2. **Number:** Must match or use defaults for missing parameters
3. **Type Relationships:** Exact match or type coercion depending on language

Parameter Passing Methods

Pass by Value: Copies actual parameter value to formal parameter. Different memory locations. Changes don't affect original.

Pass by Reference: Formal and actual parameters share same memory location. Changes affect original.

Pass by Value-Result: Copy-in at call, copy-out at return.

Pass by Name: Textual substitution, delayed evaluation (rare).

Block

Code block groups statements within delimiters (braces, begin-end keywords). Allows treating multiple statements as one and defines variable scopes.

Scoping and Accessibility

Global Scope: Variables defined outside functions, accessible everywhere, allocated at initialization

Local Scope: Variables inside functions/blocks, allocated on stack during execution, exist only during function lifetime

Name Resolution: Local variables shadow global variables within their scope

Access Levels (OOP): Public (anywhere), Private (class only), Protected (class + subclasses), Package/Internal (same package)

Abstract Data Type

ADT defines behavior through values and operations set, hiding implementation details. Specifies what operations, not how implemented.

Common ADTs: List, Stack (LIFO), Queue (FIFO), Set, Map/Dictionary

Different implementations (array/linked) possible for same ADT interface.

Generic Programming

Writing algorithms for placeholder types specified later, enabling code reuse across data types.

C++ templates, Java generics, Python type hints provide generic programming capabilities.

I/O Tools and File Handling

File Operations: Input (reading), Output (writing)

Access Modes: Read-only, Write-only, Read-write, Append

Process: Open file → Perform operations → Close file

Languages provide file objects/handles with methods like `open()`, `read()`, `write()`, `close()`.

Exception Handling

Responding to runtime errors without crashing. Try-catch blocks handle exceptions:

```
python

try:
    # risky code
except ExceptionType:
    # handle error
finally:
    # always execute
```

Common exceptions: `ArithmeticError`, `IndexError`, `NullPointerException`, `FileNotFoundException`, `IOError`

Parallel Programming

Using multiple processors/threads for simultaneous execution. Program divided into independent tasks running concurrently.

Concepts: Threads (lightweight, shared memory), Processes (independent memory), Concurrency vs Parallelism

Challenges: Race conditions, deadlocks, synchronization overhead, debugging complexity

Synchronization: Mutex, semaphore, locks, atomic operations

Modern processors have multiple cores with SMT for concurrent thread execution.

Question 2: Control Systems & Network Protocols

Synthesis of Continuous Time Control Systems

Design process: System modeling → Specifications → Controller design → Analysis → Implementation

Controllers: P (proportional), PI (proportional-integral, zero steady-state error), PID (all three actions)

Design Methods: Root locus, frequency response (Bode/Nyquist), state-space

Gain and Phase Margin

Frequency domain stability measures.

Gain Margin: Amount system gain can increase before instability. Measured at phase crossover (-180°). $GM (dB) = -20\log|G(j\omega)|$. Target: $GM > 6 \text{ dB}$

Phase Margin: Additional phase lag at gain crossover (0 dB) causing instability. $PM = 180^\circ + \angle G(j\omega)$. Target: $PM > 30^\circ$, optimal $\approx 60^\circ$

Larger margins = more robust systems. Both must be positive for stability.

Linear Systems Description

Time Domain: Differential equations, state-space ($\frac{dx}{dt} = Ax + Bu$, $y = Cx + Du$)

Frequency Domain: Transfer function $G(s) = Y(s)/X(s)$, frequency response $G(j\omega)$, Bode plots (magnitude and phase vs frequency), Nyquist plots

Linear systems obey superposition and homogeneity principles.

Signal Transfer in Control Systems

Block diagrams with transfer functions, summing junctions, signal paths.

Unity feedback closed-loop TF: $Y(s)/R(s) = G(s)/(1 + G(s))$

Connections: Cascade (multiply), Parallel (add), Feedback ($G_1/(1 \pm G_1G_2)$)

Mason's Gain Formula for complex signal flow graphs.

TCP and UDP Protocol Data Units

TCP Segment Header (20+ bytes):

- Source/Destination Port (16 bits each)
- Sequence Number (32 bits)
- Acknowledgment Number (32 bits)
- Data Offset (4 bits), Flags (9 bits including SYN, ACK, FIN)
- Window Size (16 bits)

- Checksum (16 bits)
- Options (variable)

UDP Datagram Header (8 bytes):

- Source Port (16 bits, optional)
- Destination Port (16 bits)
- Length (16 bits)
- Checksum (16 bits, optional in IPv4)

TCP: Connection-oriented, reliable (ACK+retransmission), ordered (sequence numbers), flow control, congestion control, larger header (20-60 bytes). Use: Web, email, file transfer

UDP: Connectionless, unreliable, unordered, no flow/congestion control, smaller header (8 bytes), faster. Use: Streaming, gaming, DNS, VoIP

Question 3: Digital Logic & AI Search

Combinational Logic Design

Outputs depend only on current inputs, no memory. No feedback or storage elements.

Design: Specifications → I/O determination → Truth table → Boolean simplification → Logic diagram → Verification

Multiplexers/Demultiplexers

Multiplexer: Selects one of n inputs to single output via select lines. n-input needs $\log_2(n)$ select lines.

2-to-1 MUX: $Y = S'I_0 + SI_1$

Applications: Data routing, parallel-to-serial conversion, Boolean function implementation

Demultiplexer: Routes one input to one of n outputs via select lines. Reverse of multiplexer.

Applications: Serial-to-parallel conversion, data distribution, address decoding

Encoders/Decoders

Encoder: Converts 2^n inputs to n-bit binary code. Priority encoder handles multiple active inputs.

Decoder: Converts n inputs to 2^n outputs. Only one output active per input combination.

2-to-4 Decoder: $Y_0 = A_1'A_0'$, $Y_1 = A_1'A_0$, $Y_2 = A_1A_0'$, $Y_3 = A_1A_0$

Applications: Memory address decoding, instruction decoding, Boolean function implementation

Comparators

Compares two binary numbers for magnitude relationship ($A > B$, $A = B$, $A < B$).

1-bit: $A > B = AB'$, $A = B = A \oplus \bar{B}$ (XNOR), $A < B = A'B$

n-bit: Compare bit-by-bit from MSB to LSB. Can cascade for larger numbers.

Parity Generators/Checkers

Parity Bit: Makes total 1s even (even parity) or odd (odd parity) for error detection.

Generator: $P = D_2 \oplus D_1 \oplus D_0$ (even parity)

Checker: $E = D_2 \oplus D_1 \oplus D_0 \oplus P$ ($E=0$ no error, $E=1$ error detected)

Detects odd-number bit errors, cannot detect even-number errors or correct.

Arithmetic Logic Units

Performs arithmetic (add, subtract, increment, decrement) and logical (AND, OR, NOT, XOR, shift) operations.

Components: Adder/Subtractor, Logic Unit, Shifter, MUX for operation selection

Status flags: Zero, Carry, Overflow, Negative

Problem Solving with Search

AI searches state space from initial to goal state.

Problem: Initial state, Actions, Transition model, Goal test, Path cost

Strategy properties: Completeness, Optimality, Time complexity, Space complexity

Uninformed Search

Breadth-First Search: Queue (FIFO), expands shallowest first. Complete: Yes. Optimal: Yes (uniform cost).

Time/Space: $O(b^d)$

Depth-First Search: Stack (LIFO), expands deepest first. Complete: No (infinite depth). Optimal: No. Time:

$O(b^m)$, Space: $O(bm)$

Iterative Deepening: DFS with increasing depth limits. Combines BFS completeness with DFS space efficiency.

Uniform-Cost Search: Priority queue by path cost. Expands lowest-cost node. Complete and optimal.

Heuristic Search

Uses heuristic function $h(n)$ estimating cost to goal.

Greedy Best-First: Expands lowest $h(n)$. Fast but not optimal.

A Search*: Evaluates $f(n) = g(n) + h(n)$ where $g(n)$ = cost from start, $h(n)$ = estimated cost to goal. Complete and optimal if h is admissible (never overestimates).

Admissible heuristic: $h(n) \leq h^*(n)$. Consistent: $h(n) \leq c(n,a,n') + h(n')$

Examples: Misplaced tiles, Manhattan distance (8-puzzle); Euclidean distance (pathfinding)

Constraint Satisfaction Problems

Variables X with domains D subject to constraints.

Constraint types: Unary, Binary, Higher-order, Global (Alldiff)

Backtracking Search: Try values, recursively solve, backtrack on failure.

Improvements:

- Variable ordering: MRV (minimum remaining values), Degree heuristic
- Value ordering: Least constraining value
- Constraint propagation: Forward checking, Arc consistency (AC-3)

CSP formulation enables general algorithms and efficient propagation.

Question 4: SSH & Control Principles

SSH Protocol

Secure Shell provides encrypted remote login and command execution.

Three Layers:

- Transport: Server authentication, encryption, integrity
- User Authentication: Client authentication (password, public key)
- Connection: Multiplexes logical channels

Connection Process: TCP connection → Version exchange → Algorithm negotiation → Key exchange (Diffie-Hellman) → Server authentication → User authentication → Secure session

Encryption: AES, 3DES, ChaCha20 (symmetric); Diffie-Hellman, ECDH (key exchange); HMAC-SHA (MAC)

SSH Key Generation

Public-key authentication using key pairs (private secret, public shared).

Key Types:

- RSA: Widely supported, 2048-4096 bits. `(ssh-keygen -t rsa -b 4096)`
- DSA: Deprecated, 1024-bit insufficient
- ECDSA: Shorter keys, 256-521 bits. `(ssh-keygen -t ecdsa -b 521)`
- Ed25519: Modern, fast, secure, 256-bit. `(ssh-keygen -t ed25519)` (recommended)

Private key (600 permissions, optionally passphrase-protected) proves identity. Public key added to server's `(~/.ssh/authorized_keys)`.

SSH Configuration

Client `(~/.ssh/config)`:

```
Host myserver
  HostName 192.168.1.100
  User myuser
  Port 2222
  IdentityFile ~/.ssh/id_ed25519
  ForwardAgent yes
```

Server `(/etc/ssh/sshd_config)`:

```
Port 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
```

Security: Disable root login, disable password authentication (keys only), strong key algorithms, limit user access, enable automatic disconnect for idle sessions, use fail2ban

Port Forwarding: Local (-L), Remote (-R), Dynamic/SOCKS (-D)

Control Principles

Open-Loop: Controller action independent of output. No feedback. Simple, cheap, cannot correct disturbances. Example: Timer sprinkler.

Closed-Loop (Feedback): Controller action depends on output. Measures output, compares to desired, error drives control. Compensates disturbances. Example: Thermostat.

Feedback advantages: Disturbance rejection, reduced parameter sensitivity, improved accuracy, stabilization

Feedback Control

Negative Feedback: Output subtracted from reference. Error $e(t) = r(t) - y(t)$ drives controller.

Role: Disturbance rejection, reduced parameter sensitivity, improved accuracy, bandwidth extension, stabilization

Positive Feedback: Output added to reference. Causes instability (avoided except oscillators, amplifiers).

Set Point Control and Reference Tracking

Set Point (Regulation): Maintain constant output despite disturbances. Example: Room temperature at 22°C.

Reference Tracking (Servo): Follow time-varying reference. Example: Motor position following trajectory.

Most systems perform both: regulate when reference constant, track when reference changes.

Control System Requirements

Stability: Bounded output for bounded input. Closed-loop poles have negative real parts.

Steady-State Accuracy: Zero/small steady-state error, disturbance rejection, tracking accuracy.

Transient Response: Fast settling, acceptable overshoot (<10-20%), short rise time, minimal oscillation.

Sensitivity: Low parameter variation sensitivity, robustness, noise insensitivity.

Disturbance Rejection: Minimize disturbance effects, fast recovery.

Control Effort: Reasonable actuator requirements, avoid saturation, energy efficiency.

Robustness: Maintain performance despite uncertainties, adequate margins, gradual degradation.

Requirements often conflict, requiring trade-offs.

Question 5: Game Theory & Electronics

Two-Person Games

Two players alternate moves, each pursuing their objective.

Representations:

- Extensive Form: Game tree (nodes=states, edges=moves, root=initial, leaves=terminals)
- Normal Form: Payoff matrix for simultaneous games

Properties: Zero-sum (one's gain = other's loss) vs Non-zero-sum; Perfect (complete knowledge) vs Imperfect information; Deterministic vs Stochastic

Winning Strategy and Optimal Decisions

Winning Strategy: Guaranteed win regardless of opponent moves.

Minimax Algorithm: Finds optimal play assuming perfect opponents.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL}(s) \\ \max \text{ MINIMAX}(\text{RESULT}(s,a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min \text{ MINIMAX}(\text{RESULT}(s,a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

MAX maximizes, MIN minimizes. Complete for finite games, optimal against optimal opponent. Time $O(b^m)$, Space $O(bm)$.

Alpha-Beta Pruning: Eliminates branches that cannot affect decision. α = MAX's best, β = MIN's best. Prune when $\beta \leq \alpha$. Same result as minimax, faster: $O(b^{(m/2)})$ best case.

MOS Transistor

MOSFET is fundamental digital circuit building block.

Structure: Gate (control), Source (input), Drain (output), Substrate (body)

Types:

- NMOS: N-type source/drain, P-type substrate. Conducts when $V_{GS} > V_{TH}$. Electrons as carriers.
- PMOS: P-type source/drain, N-type substrate. Conducts when $V_{SG} > |V_{TH}|$. Holes as carriers.

MOS Characteristics

Operating Regions:

- Cutoff ($V_{GS} < V_{TH}$): $I_D \approx 0$, transistor OFF
- Linear/Triode ($V_{GS} > V_{TH}$, $V_{DS} < V_{GS} - V_{TH}$): Acts as resistor, $I_D = k_n[(V_{GS} - V_{TH})V_{DS} - V_{DS}^2/2]$
- Saturation ($V_{GS} > V_{TH}$, $V_{DS} \geq V_{GS} - V_{TH}$): Current depends on V_{GS} , $I_D = (k_n/2)(V_{GS} - V_{TH})^2$

MOS as Switch

NMOS: $V_{GS}=0$ OFF, $V_{GS}=V_{DD}$ ON. Passes strong 0, weak 1. **PMOS:** $V_{SG}=0$ OFF, $V_{SG}=V_{DD}$ ON. Passes strong 1, weak 0.

Characteristics: Low R_{ON} when conducting, very high R_{OFF} when off, switching time depends on capacitance, mainly dynamic power consumption.

CMOS Inverter

Complementary MOS combines NMOS and PMOS.

Circuit: PMOS (V_{DD} to output), NMOS (output to GND), both gates connected to input.

Input=0: NMOS OFF, PMOS ON $\rightarrow$ Output= V_{DD} (High)

Input=1: NMOS ON, PMOS OFF $\rightarrow$ Output=GND (Low)

Near-zero static power, dynamic power during switching, high noise margins, sharp transition, full voltage swing.

CMOS Logic Gates

NAND: NMOS in series (both ON for Low output), PMOS in parallel

NOR: NMOS in parallel, PMOS in series (both OFF for High output)

Design Rule: NMOS network realizes F , PMOS realizes F' . NMOS pulls Low, PMOS pulls High.

Operational Amplifier

High-gain voltage amplifier with differential inputs.

Ideal Characteristics: Infinite open-loop gain, infinite input impedance (no input current), zero output impedance, infinite bandwidth, zero offset

Terminals: Non-inverting (+), Inverting (-), Output ($V_{out} = A(V_+ - V_-)$), Power supply ($\pm V_{CC}$)

Golden Rules (with negative feedback):

1. No input current: $I_+ = I_- = 0$
2. Equal input voltages: $V_+ = V_-$

Negative Feedback in Op-Amps

Connects output to inverting input. Reduces but stabilizes gain, increases bandwidth, decreases output impedance, increases input impedance, reduces distortion/noise, makes gain independent of op-amp parameters.

Basic Op-Amp Applications

Inverting Amplifier: $A_v = -R_f/R_{in}$ (negative gain, phase inversion)

Non-Inverting Amplifier: $A_v = 1 + R_f/R_{in}$ (positive gain, high input impedance)

Voltage Follower: $A_v = 1$ (buffer, impedance transformation)

Summing Amplifier: $V_{out} = -R_f(V_1/R_1 + V_2/R_2 + V_3/R_3)$ (weighted sum)

Difference Amplifier: $V_{out} = (R_f/R_{in})(V_2 - V_1)$

Integrator: $V_{out} = -(1/RC)\int V_{in} dt$ (capacitor as feedback)

Differentiator: $V_{out} = -RC(dV_{in}/dt)$ (capacitor as input)

Question 6: Sequential Logic & Web Technologies

Sequential Logic

Outputs depend on current inputs AND past history (stored state). Contains memory elements.

Types: Synchronous (state changes at clock edges), Asynchronous (immediate changes)

Latches

Level-sensitive, store one bit.

SR Latch: Set-Reset using NOR/NAND gates.

- S=0, R=0: Hold
- S=0, R=1: Reset (Q=0)
- S=1, R=0: Set (Q=1)
- S=1, R=1: Invalid

D Latch: Eliminates invalid state. Enable E: E=1 Q follows D (transparent), E=0 Hold.

Problem: Level-sensitive causes timing issues if enable high too long.

Flip-Flops

Edge-triggered, change only at clock edges.

D Flip-Flop: Master-slave configuration. At clock edge $Q \leftarrow D$. Most common type.

JK Flip-Flop:

- J=0, K=0: Hold
- J=0, K=1: Reset
- J=1, K=0: Set
- J=1, K=1: Toggle

T Flip-Flop: T=0 Hold, T=1 Toggle

Counters

Prescribed state sequence upon clock pulses.

Asynchronous (Ripple): Clock to first FF only, each triggers next. Simple but slow (cumulative delay).

Synchronous: All FFs clocked simultaneously. Faster, no cumulative delay, more complex logic.

Types: Up, Down, Up/Down, Modulo-N (0 to N-1 then reset)

Applications: Frequency division, event counting, address generation, timing

Shift Registers

Transfer data serially or parallel.

Types: SISO (serial in/out), SIPO (serial in, parallel out), PISO (parallel in, serial out), PIPO (parallel in/out)

Direction: Unidirectional or Bidirectional

Universal: All operations (SISO, SIPO, PISO, PIPO, shift left/right)

Applications: Serial communication, data conversion, arithmetic (multiply/divide by 2), pattern generation, delay lines

Memories

Store binary information in cell arrays.

Volatility: Volatile (RAM, loses data without power), Non-volatile (ROM, Flash, retains data)

RAM:

- SRAM: Flip-flop based (6 transistors), fast, no refresh, expensive, low density. Use: Cache.
- DRAM: Capacitor based (1T+1C), slower, needs refresh, cheap, high density. Use: Main memory.

ROM:

- Mask ROM: Factory programmed
- PROM: User programmable once
- EPROM: UV erasable
- EEPROM: Electrically erasable, byte-level
- Flash: Block-erasable EEPROM, high density. Use: USB drives, SSDs.

Organization: Address lines (n lines for 2^n locations), Data lines (m bits), Control lines (R/W, CS, OE)

HTML5 New Elements

Semantic Structure: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<figure>`, `<figcaption>`

Media: `<video>`, `<audio>` with controls and multiple sources

Graphics: `<canvas>` (bitmap with JavaScript), `<svg>` (vector graphics)

Forms: New input types - email, url, number, range, date, time, color, search

Other: `<details>`, `<summary>`, `<mark>`, `<progress>`, `<meter>`

CSS3 New Features

Layout: Flexbox, Grid for responsive layouts

Visual Effects:

- Border-radius (rounded corners)
- Box-shadow, text-shadow
- Gradients (linear, radial)
- Opacity, RGBA/HSLA colors

Transforms: translate, rotate, scale, skew (2D and 3D)

Transitions: Smooth property changes with duration, timing function

Animations: @keyframes for complex multi-step animations

Media Queries: Responsive design targeting devices/screen sizes

Pseudo-classes: :nth-child(), :first-of-type, :hover, :focus

Web Fonts: @font-face for custom fonts

Control Structures in Web Scripts

JavaScript provides standard control structures:

Conditional: if-else, switch-case, ternary operator

Loops: for, while, do-while, for...in (objects), for...of (iterables)

Functions: function declarations, expressions, arrow functions, callbacks

Async: Promises, async/await for asynchronous operations

Event Handling: addEventListener() for user interactions

Sensor Through Web Page

Web APIs:

- Geolocation API: GPS location
- DeviceOrientation API: Accelerometer, gyroscope
- DeviceMotion API: Motion detection
- MediaDevices API: Camera, microphone
- Ambient Light Sensor: Light level
- Battery Status API: Battery info

Access requires user permission. Data processed client-side JavaScript.

Remote Management via Web

WebSockets: Full-duplex communication for real-time control

AJAX/Fetch: Asynchronous server communication

REST APIs: HTTP methods (GET, POST, PUT, DELETE) for device control

MQTT over WebSockets: IoT messaging protocol

Server-Sent Events: Server pushes updates to browser

Web Components: Custom reusable elements for control interfaces

Security: Authentication, HTTPS/WSS, CORS policies, input validation

Question 7: Testing & Assembly Language

Software and Hardware Testing

Testing Purpose: Find defects, verify requirements, validate functionality, ensure quality

Basic Methodologies:

Black-Box: Test based on specifications without internal knowledge. Techniques: Equivalence partitioning, boundary value analysis, decision tables.

White-Box: Test based on internal structure/code. Techniques: Statement coverage, branch coverage, path coverage, condition coverage.

Gray-Box: Combination of black and white box.

Test Levels:

Unit Testing: Individual components in isolation. Developers write tests. Fast execution. Example: Testing single function.

Integration Testing: Component interactions. Tests interfaces. Example: Testing module communication.

System Testing: Complete system against requirements. End-to-end testing.

Acceptance Testing: User validation. Alpha (internal), Beta (external) testing.

Unit Testing Examples

Python (unittest):

```
python
```



```
import unittest

def add(a, b):
    return a + b

class TestMath(unittest.TestCase):
    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(add(-1, -1), -2)
```

VHDL (testbench):

```
vhdl

entity test_adder is
end test_adder;

architecture behavior of test_adder is
    signal a, b, sum : std_logic_vector(3 downto 0);
begin
    uut: adder port map(a, b, sum);

    process
    begin
        a <= "0001"; b <= "0010"; wait for 10 ns;
        assert sum = "0011" report "Test failed" severity error;
        wait;
    end process;
end behavior;
```

Hardware Testing

Functional Testing: Verify logical correctness

Timing Testing: Check setup/hold times, propagation delays

Manufacturing Testing: Detect physical defects. Stuck-at faults, bridging faults.

Techniques: Scan chains, Built-In Self-Test (BIST), boundary scan

Assembly Language Control Structures

Assembly uses jumps for control flow.

Conditional Control Transfer:

- JZ/JE: Jump if Zero/Equal
- JNZ/JNE: Jump if Not Zero/Not Equal
- JG/JGE: Jump if Greater/Greater or Equal
- JL/JLE: Jump if Less/Less or Equal
- JA/JAE: Jump if Above/Above or Equal (unsigned)
- JB/JBE: Jump if Below/Below or Equal (unsigned)

Unconditional Control Transfer:

- JMP: Unconditional jump
- CALL: Call subroutine
- RET: Return from subroutine

Branching (if-else):

```
asm

    CMP AX, BX    ; Compare
    JE equal_case ; If equal, jump
    ; else code
    JMP end_if
equal_case:
    ; equal code
end_if:
```

Cycle Organization (loops):

For loop:

```
asm

    MOV CX, 10    ; Counter
loop_start:
    ; loop body
    DEC CX        ; Decrement
    JNZ loop_start ; Jump if not zero
```

While loop:

```
asm
```

```
while_start:
    CMP AX, 0
    JLE while_end    ; Exit if AX <= 0
    ; loop body
    JMP while_start
while_end:
```

Subroutine Call:

```
asm

    CALL my_function ; Push return address, jump
    ; continues here after return

my_function:
    ; function code
    RET          ; Pop return address, jump
```

CALL pushes return address onto stack and jumps. RET pops address and jumps to it.

Question 8: Embedded Peripherals & Network Management

Typical Peripherals for Embedded Systems

UART (Universal Asynchronous Receiver-Transmitter):

- Asynchronous serial communication
- Start/stop bits frame data
- Simple, widely used
- Applications: GPS, Bluetooth modules

I2C (Inter-Integrated Circuit):

- Synchronous serial, master-slave
- Two wires: SCL (clock), SDA (data)
- Multiple devices on same bus (addressing)
- Short distances
- Applications: Sensors, EEPROMs, RTCs

SPI (Serial Peripheral Interface):

- Synchronous serial, master-slave
- Four wires: MOSI (Master Out Slave In), MISO (Master In Slave Out), SCK (Serial Clock), SS/CS (Slave Select)
- Full-duplex communication
- Higher speed than I2C
- Multiple slaves using separate select lines
- Applications: SD cards, displays, ADCs

USB (Universal Serial Bus):

- Serial communication standard
- Host-device architecture
- Hot-pluggable, self-powered options
- Various speeds: Low (1.5 Mbps), Full (12 Mbps), High (480 Mbps), SuperSpeed (5+ Gbps)
- Applications: Peripheral connectivity

GPIO (General Purpose Input/Output):

- Configurable as input or output
- Direct digital pin control
- Simple interface for switches, LEDs, sensors
- Can support interrupts

CAN (Controller Area Network):

- Multi-master serial bus
- Robust, error detection/correction
- Priority-based arbitration
- Applications: Automotive, industrial control

Single-Board Mini-Computers in Embedded Systems

Raspberry Pi:

- ARM-based processor (Cortex-A)
- Linux operating system
- GPIO pins, USB, HDMI, Ethernet

- Applications: IoT gateways, home automation, prototyping
- Advantages: Full OS, rich software ecosystem, community support
- Limitations: Not real-time, higher power consumption

Arduino:

- AVR/ARM microcontroller
- Simple programming (C/C++)
- GPIO, ADC, PWM, serial interfaces
- Applications: Sensors, actuators, simple automation
- Advantages: Ease of use, large library support
- Limitations: Limited processing power, no OS

BeagleBone:

- ARM Cortex-A processor
- Linux OS, real-time capable
- Many GPIO pins, PWM
- Applications: Industrial control, robotics
- Advantage: Real-time processing units (PRU)

Applicability: Prototyping, education, small-scale production, IoT devices, home automation, data acquisition

Network Management Systems

Functions and Services:

Fault Management: Detect, log, notify, diagnose, and correct network faults **Configuration Management:** Configure devices, backup configurations, version control **Accounting Management:** Track resource usage, billing **Performance Management:** Monitor network performance, identify bottlenecks, capacity planning **Security Management:** Control access, detect intrusions, manage encryption

SNMP (Simple Network Management Protocol):

- Standard for network monitoring and management
- Manager-agent architecture
- Operations: GET (retrieve), SET (modify), TRAP (unsolicited notification)
- MIB (Management Information Base): Database of manageable objects
- Versions: SNMPv1 (basic), SNMPv2 (improved), SNMPv3 (security)

MRTG (Multi Router Traffic Grapher):

- Monitors and graphs network traffic
- Uses SNMP to collect data from devices
- Generates HTML pages with traffic graphs
- Shows bandwidth utilization over time (daily, weekly, monthly, yearly)
- Lightweight, easy to configure
- RRDtool backend for efficient storage

Nagios:

- Comprehensive monitoring system
- Monitors hosts and services
- Alert notifications (email, SMS)
- Web-based interface
- Plugin architecture for extensibility
- Checks: Network services, host resources, environmental conditions
- Features: Event handlers, scheduled downtime, dependency management

Implementation Considerations:

- Network topology and scale
 - Monitoring requirements
 - Alert thresholds and escalation
 - Data retention policies
 - Integration with other systems
 - Scalability and performance
 - Security (authentication, encryption)
-

Question 9: FPGA & Software Engineering

Programmable Logic Devices

Programmable logic devices allow hardware configuration after manufacturing.

Types:

SPLD (Simple PLD): PAL (Programmable Array Logic), PLA (Programmable Logic Array). Limited gates.

CPLD (Complex PLD): Multiple SPLDs interconnected. Non-volatile configuration. Faster, deterministic timing. Use: Simple control logic, glue logic.

FPGA (Field-Programmable Gate Array): Most flexible, highest capacity. Volatile configuration (needs external memory). Use: Complex digital systems, signal processing, high-performance computing.

FPGA Architecture

Basic Components:

Logic Blocks (CLB - Configurable Logic Block):

- Look-Up Tables (LUTs): Implement Boolean functions (typically 4-6 inputs)
- Flip-Flops: Sequential logic, registers
- Multiplexers: Signal routing within CLB

Interconnects:

- Programmable routing connecting logic blocks
- Switch matrices at intersections
- Various wire lengths for different distances

I/O Blocks:

- Interface between internal logic and external pins
- Configurable standards (LVTTTL, LVCMOS, LVDS, etc.)
- Input/output buffers, tri-state capability

Specialized Blocks (modern FPGAs):

- Block RAM (BRAM): Fast on-chip memory
- DSP Slices: Hardware multipliers, accumulators for signal processing
- Clock Management: PLLs, DCMs for clock generation and distribution
- Hard IP Cores: PCIe, Ethernet, processors (some FPGAs)

Configuration:

- SRAM-based (volatile): Most common, fast reconfiguration
- Antifuse: One-time programmable
- Flash-based: Non-volatile

Hardware Description Languages

HDLs describe hardware structure and behavior for synthesis and simulation.

VHDL (VHSIC Hardware Description Language):

- Strongly typed
- Verbose, Pascal-like syntax
- IEEE Standard 1076
- Good for large teams, military/aerospace applications
- Example entity-architecture structure

Verilog:

- Weakly typed, C-like syntax
- More concise than VHDL
- IEEE Standard 1364
- Popular in commercial applications
- Example module structure

SystemVerilog:

- Extension of Verilog
- Enhanced verification features
- Object-oriented constructs
- IEEE Standard 1800

Common HDL Constructs:

- Concurrent assignments (dataflow)
- Process/always blocks (behavioral)
- Structural instantiation (components)
- Generate statements (parameterized design)

Designing Digital Systems in HDL

Design Flow:

1. **Specification:** Define requirements, functionality
2. **Design Entry:** Write HDL code describing circuit

3. **Simulation:** Verify functional correctness with testbenches
4. **Synthesis:** Convert HDL to netlist (gate-level representation)
5. **Implementation:** Map to FPGA resources, place and route
6. **Timing Analysis:** Verify timing constraints met
7. **Bitstream Generation:** Create configuration file
8. **Programming:** Load bitstream into FPGA
9. **Testing:** Verify hardware operation

Design Example (Simple Counter in VHDL):

```
vhdl

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity counter is
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          enable : in STD_LOGIC;
          count : out STD_LOGIC_VECTOR(7 downto 0));
end counter;

architecture Behavioral of counter is
    signal count_int : unsigned(7 downto 0);
begin
    process(clk, reset)
    begin
        if reset = '1' then
            count_int <= (others => '0');
        elsif rising_edge(clk) then
            if enable = '1' then
                count_int <= count_int + 1;
            end if;
        end if;
    end process;

    count <= std_logic_vector(count_int);
end Behavioral;
```

Testbench (simulation-only code):

```
vhdl
```

```

entity counter_tb is
end counter_tb;

architecture test of counter_tb is
    signal clk, reset, enable : std_logic;
    signal count : std_logic_vector(7 downto 0);
begin
    uut: entity work.counter
        port map(clk, reset, enable, count);

    clk_process: process
    begin
        clk <= '0'; wait for 5 ns;
        clk <= '1'; wait for 5 ns;
    end process;

    stim_process: process
    begin
        reset <= '1'; wait for 20 ns;
        reset <= '0'; enable <= '1';
        wait;
    end process;
end test;

```

FPGA Implementation Considerations:

- Clock domain crossing
- Timing constraints
- Resource utilization (LUTs, FFs, BRAM, DSP)
- Power consumption
- I/O standards and pin assignments
- Metastability prevention

Software Engineering Concepts

Software engineering applies systematic approaches to software development.

Different Paradigms:

Procedural: Focus on procedures/functions (C) **Object-Oriented:** Focus on objects and classes (Java, C++)

Functional: Focus on mathematical functions (Haskell, Lisp) **Logic:** Focus on logical relations (Prolog)

Classical Methodologies

Waterfall (Structured) Model: Sequential phases, each completed before next begins.

Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance

Characteristics:

- Rigid structure, difficult to change requirements
- Extensive documentation
- Well-defined milestones

Advantages:

- Simple, easy to understand
- Well-defined stages
- Works for stable requirements

Disadvantages:

- Inflexible to changes
- Late testing, late risk discovery
- Customer sees product late
- Not suitable for complex, evolving projects

Iterative Models

Evolutionary Model:

- System developed in increments
- Each increment adds functionality
- Feedback incorporated in next increment
- Continuous refinement

Incremental Model:

- System divided into modules
- Each module developed and delivered separately
- Early modules operational while later developed
- Reduces initial delivery time

Spiral Model:

- Combines iterative and waterfall
- Risk-driven approach
- Phases: Planning → Risk Analysis → Engineering → Evaluation
- Each spiral adds functionality
- Suitable for large, risky projects

Advantages (Iterative):

- Flexible to changes
- Early delivery of working software
- Risk reduction through iterations
- Customer feedback incorporated

Disadvantages:

- Requires good planning
- May lack documentation
- Resource intensive

Agile Software Development

Agile emphasizes flexibility, customer collaboration, iterative development.

Agile Manifesto Values:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Methodologies:

Scrum:

- Sprints (2-4 weeks) deliver increments
- Roles: Product Owner, Scrum Master, Development Team
- Ceremonies: Sprint Planning, Daily Standup, Sprint Review, Retrospective
- Artifacts: Product Backlog, Sprint Backlog, Increment

Extreme Programming (XP):

- Pair programming
- Test-Driven Development (TDD)
- Continuous Integration
- Refactoring
- Simple design
- Short releases

Kanban:

- Visualize workflow
- Limit work in progress
- Continuous flow
- Pull system

Agile Tools:

- Version Control: Git, SVN
- CI/CD: Jenkins, GitLab CI, GitHub Actions
- Project Management: Jira, Trello, Azure DevOps
- Communication: Slack, Microsoft Teams

Advantages:

- Rapid delivery, flexibility
- Customer satisfaction
- Quality focus (continuous testing)
- Team collaboration

Disadvantages:

- Requires experienced team
- Documentation may be insufficient
- Scope creep risk
- Less predictable timeline/cost

OO Design Principles (SOLID)

Single Responsibility: Class has one reason to change **Open/Closed:** Open for extension, closed for modification **Liskov Substitution:** Subtypes must be substitutable for base types **Interface Segregation:** Many specific interfaces better than one general **Dependency Inversion:** Depend on abstractions, not concretions

Important Design Patterns

Creational: Singleton, Factory, Builder, Prototype **Structural:** Adapter, Bridge, Composite, Decorator, Facade **Behavioral:** Observer, Strategy, Command, Iterator, State

MVC Architectural Pattern

Model-View-Controller separates concerns:

Model: Data and business logic. Notifies views of changes. **View:** Presentation layer. Displays model data. **Controller:** Handles user input. Updates model, selects view.

Flow: User interacts with View → Controller processes → Model updates → View refreshes

Benefits:

- Separation of concerns
- Multiple views for same model
- Easier testing
- Parallel development

Applications: Web applications (Django, Ruby on Rails), desktop applications, mobile apps

Question 10: SSL/OpenSSL & x86 Architecture

Web Server Configuration Using SSL

SSL/TLS (Secure Sockets Layer/Transport Layer Security) provides secure communication.

TLS Versions: TLS 1.2 (minimum), TLS 1.3 (current, recommended)

How TLS Works:

1. **Client Hello:** Client sends supported cipher suites, TLS versions
2. **Server Hello:** Server selects cipher suite, sends certificate
3. **Certificate Verification:** Client validates server certificate
4. **Key Exchange:** Establish shared secret (RSA, Diffie-Hellman, ECDHE)
5. **Finished:** Both send encrypted messages to verify

6. Secure Communication: Symmetric encryption for data

Certificate Components:

- Public key
- Identity information (domain, organization)
- Issuer (Certificate Authority)
- Validity period
- Signature

Configuring Apache with SSL:

```
apache

<VirtualHost *:443>
    ServerName example.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /path/to/certificate.crt
    SSLCertificateKeyFile /path/to/private.key
    SSLCertificateChainFile /path/to/chain.crt

    SSLProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1
    SSLCipherSuite HIGH:!aNULL:!MD5
    SSLHonorCipherOrder on
</VirtualHost>
```

Nginx with SSL:

```
nginx

server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /path/to/certificate.crt;
    ssl_certificate_key /path/to/private.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;
}
```

OpenSSL Cryptographic Library

OpenSSL provides cryptographic functions for SSL/TLS.

Basic Functions:

Authentication:

- Certificate generation and management
- Public/Private key operations
- Digital signatures
- Certificate Signing Requests (CSR)

Encryption:

- Symmetric: AES, DES, 3DES, RC4, ChaCha20
- Asymmetric: RSA, DSA, ECDSA, Ed25519
- Hashing: SHA-256, SHA-384, SHA-512, MD5 (deprecated)

Key Operations:

Generate private key:

```
bash  
openssl genrsa -out private.key 4096
```

Generate CSR:

```
bash  
openssl req -new -key private.key -out request.csr
```

Self-signed certificate:

```
bash  
openssl req -x509 -new -nodes -key private.key -sha256 -days 365 -out certificate.crt
```

View certificate:

```
bash  
openssl x509 -in certificate.crt -text -noout
```


Test SSL connection:

```
bash

openssl s_client -connect example.com:443
```

Encryption/Decryption:

```
bash

# Encrypt file
openssl enc -aes-256-cbc -salt -in file.txt -out file.enc

# Decrypt file
openssl enc -d -aes-256-cbc -in file.enc -out file.txt
```

Hashing:

```
bash

openssl dgst -sha256 file.txt
```

Security Best Practices:

- Use strong keys (RSA 4096, ECDSA 384)
 - Disable weak ciphers and protocols
 - Use Perfect Forward Secrecy (ECDHE)
 - Implement HSTS (HTTP Strict Transport Security)
 - Regular certificate renewal
 - Secure private key storage (600 permissions)
-

Intel x86 Instruction Set Architecture

Registers:

General Purpose Registers (64-bit, can access 32/16/8-bit):

- RAX/EAX/AX/AL: Accumulator, arithmetic
- RBX/EBX/BX/BL: Base, addressing
- RCX/ECX/CX/CL: Counter, loops
- RDX/EDX/DX/DI: Data, I/O

- RSI/ESI/SI: Source Index
- RDI/EDI/DI: Destination Index
- RBP/EBP/BP: Base Pointer, stack frame
- RSP/ESP/SP: Stack Pointer
- R8-R15: Additional 64-bit registers

Segment Registers:

- CS: Code Segment
- DS: Data Segment
- SS: Stack Segment
- ES, FS, GS: Extra Segments

Control Registers:

- RIP/EIP/IP: Instruction Pointer
- RFLAGS/EFLAGS: Flags (Zero, Carry, Sign, Overflow, etc.)

Addressing Modes:

Immediate: Value in instruction (`MOV EAX, 5`) **Register:** Value in register (`MOV EAX, EBX`) **Direct:** Memory address (`MOV EAX, [1000h]`) **Register Indirect:** Address in register (`MOV EAX, [EBX]`) **Based:** Base + displacement (`MOV EAX, [EBX+4]`) **Indexed:** Base + Index*Scale (`MOV EAX, [EBX+ECX*4]`) **Based Indexed:** Base + Index + displacement (`MOV EAX, [EBX+ECX+8]`)

Instruction Categories:

Data Movement: MOV, PUSH, POP, XCHG, LEA

Arithmetic: ADD, SUB, INC, DEC, MUL, DIV, NEG

Logical: AND, OR, XOR, NOT, TEST

Shift/Rotate: SHL, SHR, SAL, SAR, ROL, ROR

Control Flow: JMP, CALL, RET, Conditional Jumps (JE, JNE, JG, JL, etc.)

String: MOVS, CMPS, SCAS, LODS, STOS

System: INT (interrupt), SYSENTER, SYSEXIT

Memory Architecture

Segmentation (legacy, mostly flat in 64-bit):

- Segments: Code, Data, Stack

- Segment selectors + offset → Linear address

Paging:

- Virtual to physical address translation
- Page size: 4KB (typical), 2MB, 1GB (huge pages)
- Page Tables: Hierarchical structure
- TLB (Translation Lookaside Buffer): Cache for translations

64-bit Paging (4-level):

- PML4 (Page Map Level 4)
- PDPT (Page Directory Pointer Table)
- PD (Page Directory)
- PT (Page Table)

Memory Hierarchy:

- Registers (fastest, smallest)
- L1 Cache (per core, ~32-64KB)
- L2 Cache (per core, ~256KB-1MB)
- L3 Cache (shared, several MB)
- RAM (GB)
- Disk (slowest, largest)

Interrupt System

Interrupts disrupt normal execution to handle events.

Types:

Hardware Interrupts: External devices (keyboard, timer, disk) **Software Interrupts:** INT instruction

Exceptions: CPU-detected errors (divide by zero, page fault)

Interrupt Vector Table (IVT) / Interrupt Descriptor Table (IDT):

- Table mapping interrupt numbers to handler addresses
- 256 entries (0-255)
- Protected mode uses IDT with descriptors

Common Interrupts:

- INT 0: Divide Error
- INT 1: Debug
- INT 3: Breakpoint
- INT 13: General Protection Fault
- INT 14: Page Fault
- INT 32-255: User/device interrupts

Interrupt Handling:

1. Save context (flags, CS, IP)
2. Disable interrupts (IF=0)
3. Jump to handler (via IDT)
4. Execute handler
5. IRET (restore context, return)

Programmable Interrupt Controller (PIC/APIC):

- Manages hardware interrupts
 - Priority levels
 - Masking individual interrupts
-

Question 11: IPC & Algorithms

Inter-Process Communication Methods

IPC enables processes to exchange data and synchronize.

File-Based IPC:

- Processes read/write shared files
- Simple but slow (disk I/O)
- Needs synchronization (file locking)
- Example: Configuration files, logs

Signals:

- Asynchronous notifications
- Limited information (signal number only)

- Common signals: SIGTERM (terminate), SIGKILL (force kill), SIGUSR1/2 (user-defined), SIGCHLD (child status change)
- Handler functions process signals
- Example: `kill -SIGTERM pid`

Pipes:

- Unidirectional data channel
- **Anonymous pipes:** Related processes (parent-child). `pipe()` system call. Example: shell commands `ls | grep txt`
- **Named pipes (FIFOs):** Unrelated processes. Created with `mkfifo`. Persistent in filesystem.
- Data stream, no message boundaries
- Blocking I/O (unless non-blocking mode)

Sockets:

- Bidirectional communication
- **Unix Domain Sockets:** Same machine, fast
- **Internet Sockets:** Network communication (TCP/UDP)
- Supports various protocols
- Socket API: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`, `close()`
- Most flexible and powerful IPC

Other IPC Methods (not in question but relevant):

- **Shared Memory:** Fastest, processes map same memory region
- **Message Queues:** Message-based, asynchronous
- **Semaphores:** Synchronization primitive

Comparison:

Method	Speed	Directionality	Scope	Use Case
File	Slow	Bidirectional	System-wide	Config, logs
Signal	Fast	Unidirectional	Process	Notifications
Pipe	Fast	Unidirectional	Related	Command chains
Socket	Medium	Bidirectional	Network	Client-server

Algorithm Complexity

Describes resource requirements (time, space) as input size grows.

Asymptotic Notations:

Big-O (O): Upper bound, worst-case. $O(f(n))$: Algorithm takes at most $c \cdot f(n)$ time for large n .

Big-Omega (Ω): Lower bound, best-case. $\Omega(f(n))$: Algorithm takes at least $c \cdot f(n)$ time.

Big-Theta (Θ): Tight bound, average-case. $\Theta(f(n))$: Algorithm takes exactly $c \cdot f(n)$ time (within constants).

Common Complexities (from fastest to slowest):

- $O(1)$: Constant
- $O(\log n)$: Logarithmic (binary search)
- $O(n)$: Linear (linear search)
- $O(n \log n)$: Linearithmic (merge sort, quick sort average)
- $O(n^2)$: Quadratic (bubble sort, insertion sort)
- $O(n^3)$: Cubic (naive matrix multiplication)
- $O(2^n)$: Exponential (recursive Fibonacci)
- $O(n!)$: Factorial (traveling salesman brute force)

Rules:

- Drop constants: $O(2n) = O(n)$
- Drop lower terms: $O(n^2 + n) = O(n^2)$
- Different inputs use different variables: $O(a + b)$, $O(a \cdot b)$

Insertion Sort

Simple sorting algorithm, efficient for small or nearly-sorted data.

Algorithm:

python

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```

Complexity:

- Best case: $O(n)$ - already sorted
- Average/Worst case: $O(n^2)$
- Space: $O(1)$ - in-place

Characteristics: Stable, adaptive, simple, good for small/nearly-sorted data.

Searching

Linear Search:

- Check each element sequentially
- Time: $O(n)$
- Works on unsorted data

Binary Search:

- Divide and conquer on sorted data
- Compare middle element, eliminate half
- Time: $O(\log n)$
- Requires sorted array

python

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Hash Tables

Data structure for fast key-value lookup.

Hash Function: Maps key to index: $\text{index} = \text{hash}(\text{key}) \% \text{table_size}$

Properties of Good Hash Function:

- Deterministic
- Uniform distribution
- Fast computation
- Minimal collisions

Collision Resolution:

Chaining: Each table entry is linked list. Multiple keys at same index stored in list.

- Simple, handles many collisions
- Extra memory for pointers
- $O(1)$ average, $O(n)$ worst case

Open Addressing: Find alternative location.

- **Linear Probing:** Check next slot: $(\text{hash} + i) \% \text{size}$
- **Quadratic Probing:** $(\text{hash} + i^2) \% \text{size}$
- **Double Hashing:** $(\text{hash1} + i \cdot \text{hash2}) \% \text{size}$
- No extra memory
- Clustering issues

Complexity:

- Average: $O(1)$ for insert, search, delete
- Worst: $O(n)$ - all keys collide

Load Factor: $\alpha = n/m$ (items/table size). Keep $\alpha < 0.7$ for good performance.

Graphs

Representation:

Adjacency Matrix: 2D array, $matrix[i][j]$ = edge weight/existence

- Space: $O(V^2)$
- Edge check: $O(1)$
- Iteration: $O(V^2)$
- Good for dense graphs

Adjacency List: Array of lists, $list[i]$ contains neighbors of vertex i

- Space: $O(V + E)$
- Edge check: $O(\text{degree})$
- Iteration: $O(V + E)$
- Good for sparse graphs

Breadth-First Search (BFS):

- Explores level by level
- Uses queue
- Finds shortest path in unweighted graph
- Time: $O(V + E)$, Space: $O(V)$

```
python
```

```
def bfs(graph, start):
    visited = set()
    queue = [start]
    visited.add(start)

    while queue:
        vertex = queue.pop(0)
        print(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Depth-First Search (DFS):

- Explores as deep as possible first
- Uses stack (or recursion)
- Detects cycles, topological sort
- Time: $O(V + E)$, Space: $O(V)$

```
python

def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start)

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

Applications:

- BFS: Shortest path, level-order traversal, connected components
 - DFS: Cycle detection, topological sort, strongly connected components, maze solving
-

Question 12: Databases & Power Electronics

Entity-Relationship Model

ER model designs databases conceptually before implementation.

Components:

Entity: Object or thing (person, product, order). Represented as rectangle.

Attribute: Property of entity (name, price, date). Represented as oval. Types:

- **Simple:** Atomic, indivisible (age)
- **Composite:** Subdivided (address = street + city + zip)
- **Derived:** Calculated from others (age from birthdate)
- **Multivalued:** Multiple values (phone numbers)
- **Key:** Uniquely identifies entity (underlined)

Relationship: Association between entities. Represented as diamond. Can have attributes.

Cardinality:

- One-to-One (1:1): Person has one passport
- One-to-Many (1:N): Department has many employees
- Many-to-Many (M:N): Students enroll in courses

Participation:

- Total (double line): Every entity participates
- Partial (single line): Some entities participate

Weak Entity: Depends on strong entity for identification. Double rectangle, identifying relationship double diamond.

ER Diagram Example: University Database

- Entities: Student, Course, Department, Professor
- Relationships: Enrolls (Student-Course, M:N), Teaches (Professor-Course, 1:N), Belongs (Student-Department, N:1)

Relational Data Model

Relation: Table with rows and columns.

Scheme: Structure of relation: Name(Attr1, Attr2, ...)

Attribute: Column in table.

Tuple: Row in table, instance of relation.

Domain: Set of allowed values for attribute.

Keys:

- **Super Key:** Attributes uniquely identifying tuple
- **Candidate Key:** Minimal super key
- **Primary Key:** Selected candidate key (underlined)
- **Foreign Key:** References primary key in another relation

Constraints:

- Entity Integrity: Primary key cannot be NULL
- Referential Integrity: Foreign key must exist in referenced table or be NULL
- Domain Constraint: Values must be from domain

Building Relational Schema from ER Diagram

Rules:

1. **Strong Entity** $\rightarrow$ Relation
 - Attributes become columns
 - Key attribute becomes primary key
2. **Weak Entity** $\rightarrow$ Relation
 - Include own attributes + primary key of strong entity (foreign key)
 - Composite primary key: weak entity's partial key + foreign key
3. **1:1 Relationship** $\rightarrow$ Add foreign key to either side (prefer total participation side)
4. **1:N Relationship** $\rightarrow$ Add foreign key on N side
5. **M:N Relationship** $\rightarrow$ New relation
 - Primary keys from both entities as foreign keys
 - Composite primary key
 - Relationship attributes added
6. **Multivalued Attribute** $\rightarrow$ New relation
 - Entity's primary key + multivalued attribute
 - Composite primary key

Example: ER: Student(StudentID, Name) -- Enrolls(Grade) -- Course(CourseID, Title) (M:N relationship)

Relational:

- Student(StudentID, Name)
- Course(CourseID, Title)
- Enrolls(StudentID, CourseID, Grade)
 - StudentID, CourseID are foreign keys
 - Primary key: (StudentID, Course